

- **Insecure dependencies:** Outdated or poorly maintained npm packages can introduce vulnerabilities. Regular audits using tools like NPM Audit or Snyk are essential.
 - **Injection attacks:** Malicious data sent to interpreters, like SQL or shell injection, can compromise systems. Parameterized queries and input validation must be used.
- For Node.js applications, managing environment variables securely, validating data rigorously, and using middleware like Helmet for HTTP headers protection are crucial best practices. Logging and monitoring for unusual activities also play a vital role.
- Security must be a continuous process, integrated into development,

deployment, and maintenance phases.

JavaScript and Node.js: The future of Web innovation

JavaScript and Node.js have not only sustained their relevance but continue to lead the charge in modern web development. Their ubiquity, adaptability, and developer-friendly ecosystems make them prime tools for innovation in the digital age.

From simple web interactions to complex server-side processing, and now AI, VR, and real-time communication, JavaScript is pushing the boundaries of what's possible on the web. Node.js, by enabling JavaScript on the server, has empowered full-stack development, accelerated project delivery, and supported high-performance applications.

The synergy between JavaScript's evolving standards (like ECMAScript updates), vibrant community, and growing cross-platform capabilities ensures that it remains a dominant force. As technologies like WebAssembly, Edge Computing, and Web3 mature, JavaScript is expected to integrate and evolve with them.

For developers, startups, and enterprises alike, investing in JavaScript and Node.js is investing in the future of the web. The journey is far from over—in fact, it's just getting started. **END** 🐧

 **By: Supritha R.S.**

The author is a software engineer and tech enthusiast, driven by a fascination for cloud technology and distributed database systems.

■■ Continued from page...47

Security remains a paramount concern in modern technology, and Java 24 addresses this with robust security enhancements. The latest version includes updates to cryptographic libraries, better support for secure coding practices, and more stringent access controls, ensuring that Java applications can withstand emerging security threats.

Additionally, Java 24 aims to streamline the development experience through tooling improvements and better integration with modern development environments. Enhanced support for containerization and cloud-native development positions Java as a viable option for building scalable, distributed systems that leverage cloud infrastructures.

Furthermore, Java 24's commitment to backward compatibility ensures that existing Java applications can seamlessly transition to the new version without extensive refactoring. This stability allows organisations to adopt the latest advancements without disrupting their current workflows.

Java's enduring relevance in modern technology is a testament to its foundational strengths and continuous evolution. Java 24, with its focus on performance,

security, and developer experience, is well-equipped to compete with other programming languages in developing new age solutions. As the technology landscape continues to advance, Java's adaptability and robust ecosystem will undoubtedly play a crucial role in shaping the future of software development. **END** 🐧

References

- <https://openjdk.org/projects/jdk/24/>
- <https://www.infoworld.com/article/3491404/jdk-24-the-new-features-in-java-24.html>
- <https://www.oracle.com/in/news/announcement/oracle-releases-java-24-2025-03-18/>

 **By: Dr Magesh Kasthuri and Muthu Vinayagam**

Dr Magesh Kasthuri is a distinguished member of technical staff and chief architect in Wipro Limited.

Muthu Vinayagam is an application architect and Java expert working at Wipro Limited in the US.

Disclaimer: This article expresses the views of the authors and not that of Wipro.